

Flight Software Engineer Assessment

VyomIC – Mission Planning Group (MPG)

Overview

You have recently joined VyomIC as a Flight Software Engineer responsible for the payload software of a next-generation LEO Positioning, Navigation and Timing (LEO-PNT) satellite constellation operating at an altitude of approximately **1000 km**. The spacecraft has completed subsystem-level software development and is approaching its **Critical Design Review (CDR)** before integration with the satellite bus.

Your responsibility is to define the software architecture that governs payload operations, software modes, subsystem interaction, housekeeping telemetry, and fault handling during nominal spacecraft operation.

The payload consists of:

- GNSS Receiver
- Orbit Determination Software
- Payload FPGA
- High Stability Oscillator
- Onboard Computer (Linux-based)
- IMU
- X-band Downlink

As part of this exercise, you are expected to think as a Flight Software Engineer rather than an algorithm developer. Justify every design decision you make.

Task 1: Flight Software Architecture & Operational Modes Analysis

Design the high-level software architecture for the payload subsystem. Your answer should include:

- Major software modules
- Responsibilities of each module
- Communication between modules
- External interfaces with the spacecraft bus
- External interfaces with the ground segment

Present the architecture using a block diagram accompanied by a brief explanation and, using the architecture designed above, define the operational software state machine for the payload. Include (considering entry, exit and operations):

- BOOT
- SAFE
- COMMISSIONING
- NOMINAL
- DIAGNOSTIC

Task 2: Housekeeping Telemetry and Task Scheduling

Assume that the payload software periodically monitors subsystem health while simultaneously processing GNSS measurements. Design:

- A housekeeping telemetry packet suitable for nominal operations as per CCSDS norms.
- A software scheduling strategy for all periodic and event-driven tasks.

Clearly justify:

- Task priorities
- Scheduling methodology
- Housekeeping update rates
- Event-driven versus periodic telemetry

Task 3: Fault Detection and Recovery Strategy

During nominal operation, the GNSS receiver unexpectedly stops producing measurements while the payload CPU utilisation begins increasing steadily.

Design an FDIR strategy describing:

- Fault detection
- Fault isolation
- Automatic recovery actions
- Transition criteria to SAFE mode
- Conditions for autonomous return to NOMINAL mode

Clearly distinguish between software and hardware faults.

Task 4: Coding Exercise

Implement a **Housekeeping Monitor** in a coding language of your choice.

The program shall:

- Read the JSON telemetry file sent via email.
- Parse housekeeping packets.
- Detect configurable threshold violations.
- Generate event messages with four severity levels:
 - INFO
 - WARNING
 - ERROR
 - CRITICAL
- Produce a concise event log indicating:
 - Timestamp
 - Subsystem
 - Violated parameter
 - Severity
 - Recommended software action

Your implementation should be modular, readable, and easily extensible to additional telemetry parameters.

Task 5: Engineering Justification

Prepare a note discussing the key design trade-offs you made throughout this assessment. Examples may include:

- Simplicity versus robustness
- Fault tolerance versus operational continuity
- Memory usage versus logging capability
- Modular architecture versus execution efficiency
- Periodic versus event-driven software

Support your decisions with engineering reasoning rather than personal preference.